AI AGENT

Complete Developer Study Notes

Concepts • Examples • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to AI Agents

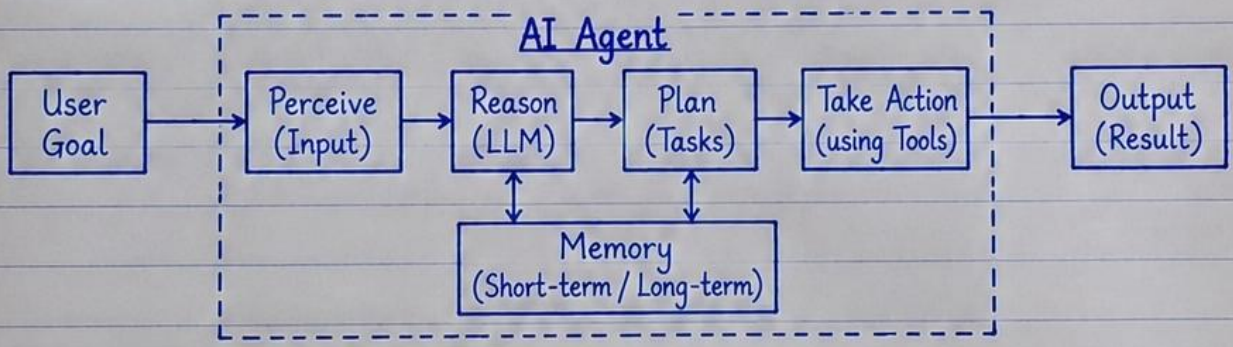
1.1 What is an AI Agent?

→ An AI Agent is an intelligent system that can perceive, reason, plan and take actions to achieve a goal.
→ It can interact with tools, external systems and environment to complete tasks automatically.
→ AI Agents use LLMs + Tools + Memory + Planning to perform complex real-world tasks.

1.2 Key Characteristics

① Autonomous → Can work with minimal human intervention
② Goal-Oriented → Focuses on achieving a specific goal
③ Uses Tools → Can call APIs, databases, files, browsers etc.
④ Memory → Remembers past interactions and context
⑤ Reasoning → Plans and decides next steps using LLM

1.3 How AI Agent Works?



1.4 Real World Examples

→ Personal AI Assistant (email, calendar, meeting tasks)
→ Research Agent (search, read, summarize)
→ Coding Agent (write code, run, debug, deploy)
→ Data Analysis Agent (query DB, analyze, generate report)
→ Customer Support Agent (answer, retrieve info, take actions)

1.5 Benefits

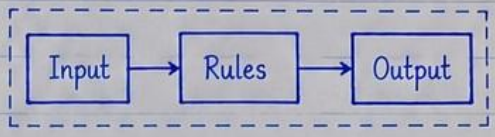
→ Increases productivity
→ Automates complex tasks
→ Can work 24/7
→ Reduces manual effort
→ Useful in real-world business applications

Rahul Yadav

2. Types of AI Agents

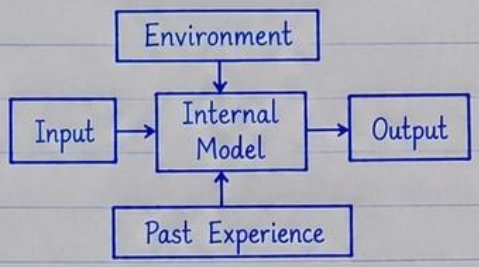
2.1 Rule-based Agent (Simple Reflex Agent)

- → Works on predefined rules and conditions.
- → Responds to current input only.
- → Does not have memory or learning capability.
- → Suitable for simple and well-defined tasks.
- → Example: Chatbot with fixed responses, automation scripts.



2.2 Model-based Reflex Agent

- → Maintains an internal model of the environment.
- → Considers past states to make better decisions.
- → Still works on rules, but with internal state/memory.
- → Useful in partially observable environments.
- → Example: Robot navigation, game agents.



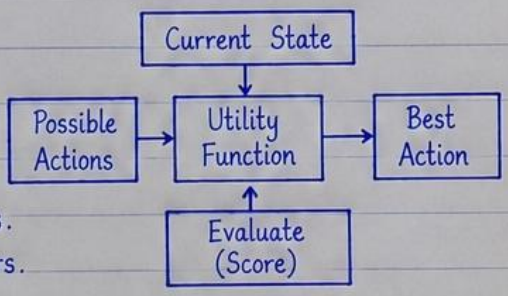
2.3 Goal-based Agent

- → Takes actions to achieve a specific goal.
- → Evaluates different possible actions.
- → Uses search and planning techniques.
- → Can handle multiple possible paths.
- → Example: Route planning, task automation.



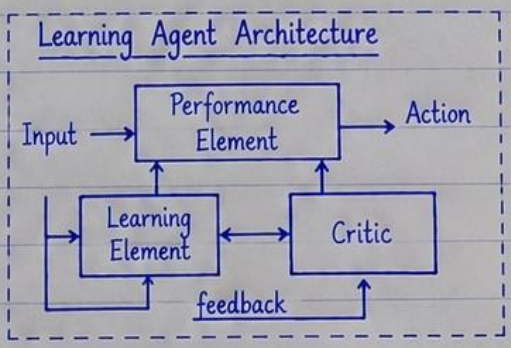
2.4 Utility-based Agent

- → Uses a utility function to choose the best action.
- → Considers multiple goals and trade-offs.
- → Chooses actions that maximize overall utility.
- → Better than goal-based agent for complex scenarios.
- → Example: Recommendation systems, self-driving cars.



2.5 Learning Agent

- → Learns from past experience and improves over time.
- → Uses techniques like Machine Learning, RL etc.
- → Can adapt to changing environments.
- → Has components like learning, performance and critic.
- → Example: Personal AI assistants, adaptive systems.



2.6 Comparison

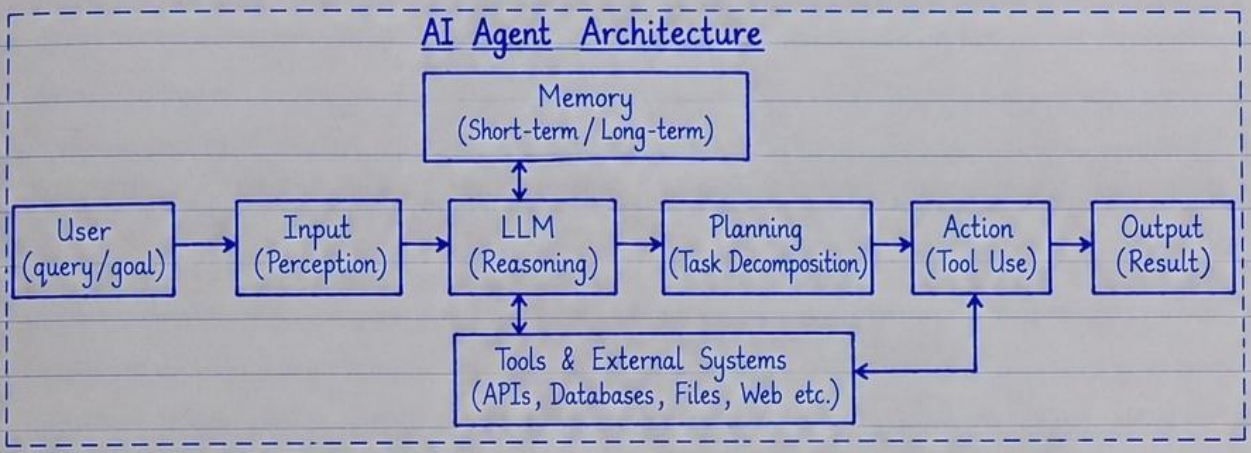
Type	Memory	Learning	Goal	Complexity	Example
Rule-based	No	No	No	Low	Chatbot
Model-based	Yes	No	No	Medium	Robot navigation
Goal-based	Yes	No	Yes	Medium	Route planning
Utility-based	Yes	No	Yes (Multiple)	High	Recommendation
Learning Agent	Yes	Yes	Yes	High	AI Assistant

Rahul Yadav

3. Agent Architecture & Components

3.1 High Level Architecture

→ An AI Agent consists of multiple components that work together to perceive, reason, plan, take actions and learn from feedback.
→ It interacts with external tools, APIs, databases and environment.
→ Architecture can vary based on the type of agent (rule-based, model-based, goal-based, etc).



3.2 Components Explained

① **Input / Perception**
→ Understands user query, context and environment.
→ Can handle text, documents, images, etc.

② **LLM (Reasoning)**
→ Uses language model to think, reason and generate responses.
→ Can follow instructions and use tools.

③ **Planning**
→ Breaks complex goal into smaller tasks.
→ Decides which tools to use and in what order.

④ **Action / Tool Use**
→ Executes actions using external tools (APIs, databases, code, files etc).
→ Can call multiple tools in sequence.

⑤ **Memory**
→ Stores past interactions, context and learnings.
→ Two types: Short-term (conversation) and Long-term (persistent).

⑥ **Output**
→ Returns final result to user.
→ Can be text, file, report, action result, etc.

3.3 Types of Memory

→ **Short-term Memory**
- Stores recent conversation context
- Helps in maintaining continuity

→ **Long-term Memory**
- Stores important information for future
- Can be vector database, knowledge base etc.

→ **Working Memory**
- Temporary memory for current task
- Used while planning and reasoning

3.4 Tools & External Systems

→ Web search (Google, Bing)
→ APIs (REST, GraphQL)
→ Databases (SQL, MongoDB, Vector DB)
→ File system (read/write files)
→ Code execution (Python, shell)
→ Third-party services (Slack, Email, etc.)

3.5 Key Points

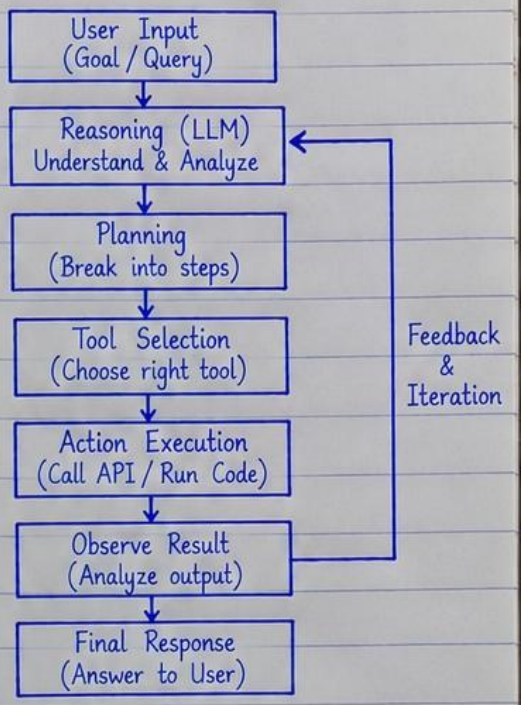
→ Agents are modular and extensible.
→ LLM is the brain, tools are the hands.
→ Memory and planning improve performance.
→ Architecture can be customized as per use case.

Rahul Yadav

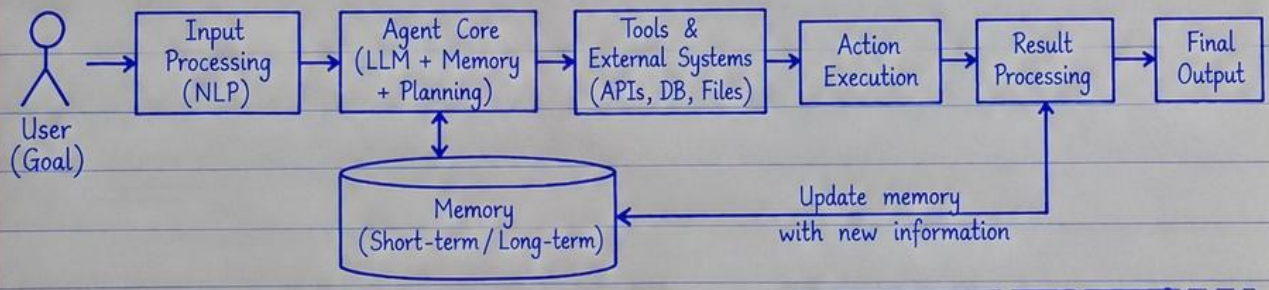
4. How AI Agents Work

4.1 Step by Step Flow

1. Understand the goal (user input)
2. Analyze and reason using LLM
3. Plan the steps (task decomposition)
4. Select and use appropriate tools
5. Execute actions (call APIs, run code etc.)
6. Observe the results
7. Use memory to learn and improve
8. Return final response to the user.

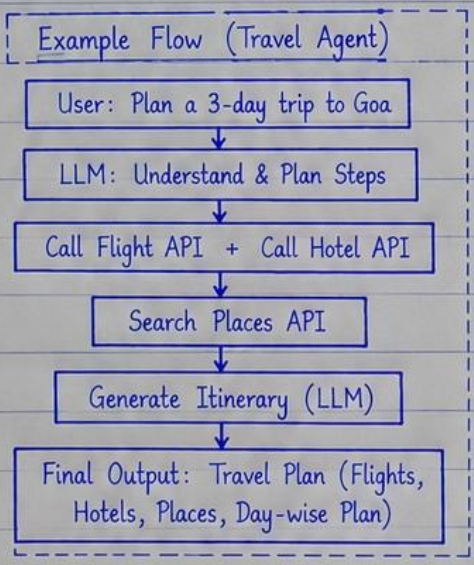


4.2 Detailed Workflow



4.3 Example: Travel Planning Agent

1. User Input: Plan a 3-day trip to Goa with flight, hotel and sightseeing details.
2. Agent understands the requirement.
3. Plans the steps: search flights → search hotels → find places → create itinerary.
4. Uses tools: Flight API, Hotel API, Search API.
5. Collects results and analyzes options.
6. Creates final travel plan with recommendations.
7. Returns a structured response to the user.



4.4 Key Points

- → AI Agents work in a loop (Plan → Act → Observe → Improve).
- → They use LLM for reasoning and decision making.
- → Tools and external systems help in taking real actions.
- → Memory enables better context and personalization.
- → Agents can handle complex, multi-step tasks automatically.

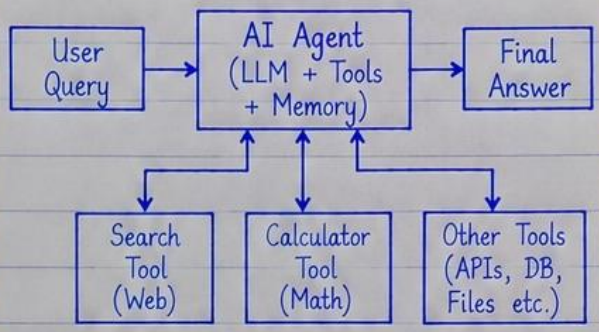
Rahul Yadav

5. AI Agent Example with Python

5.1 What we will build ?

→ A simple AI Agent using Python.
→ The agent can take a user query, use tools (e.g. web search, calculator), and give a final answer.
→ We will use OpenAI LLM and LangChain (tool calling) for this example.

High Level Architecture



5.2 Setup and Installation

Install required libraries:

```
pip install openai langchain duckduckgo-search
```

5.3 Create Tools

① Web Search Tool (DuckDuckGo)

```
from langchain.tools import DuckDuckGoSearchRun
search_tool = DuckDuckGoSearchRun()
```

② Calculator Tool

```
from langchain.tools import Tool
import math

def calculate(expression: str) → str:
    try:
        result = eval(expression, {"__builtins__": {}},
                      {"math": math})
        return str(result)
    except Exception as e:
        return f"Error: {str(e)}"

calculator_tool = Tool(
    name = "Calculator",
    func=calculate,
    description="Useful for mathematical calculations"
)
```

5.4 Create the AI Agent

```
from langchain.chat_models import ChatOpenAI
from langchain.agents import initialize_agent
from langchain.agents import AgentType

tools = [search_tool, calculator_tool]

llm = ChatOpenAI(model="gpt-3.5-turbo",
                 temperature=0)

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
```

5.5 Run the Agent

```
query = "What is the population of India?
         Also calculate 12 * 34 and tell me
         the latest news about space mission."

response = agent.run(query)
print(response)
```

5.6 Sample Output (Simplified)

→ Enter Query: What is 12 * 34 ?
→ Thought: I should use calculator tool.
→ Action: Calculator
→ Action Input: 12 * 34
→ Observation: 408
→ Final Answer: The result of 12 * 34 is 408.

5.7 Key Points

→ We can add multiple tools (search, calculator, APIs, DB, files etc.).
→ LLM decides which tool to use based on the query.
→ This is a simple example. In real applications we can add memory, better prompts, error handling and custom tools.
→ Frameworks like LangChain and LlamaIndex make it easier to build advanced agents.

Rahul Yadav

6. Memory in AI Agents

6.1 Why Memory is Important?

- → Helps agent remember past interactions.
- → Maintains conversation context.
- → Avoids repeating same questions.
- → Enables personalized responses.
- → Useful for multi-step and long tasks.
- → Without memory, agent treats each query as a new request.

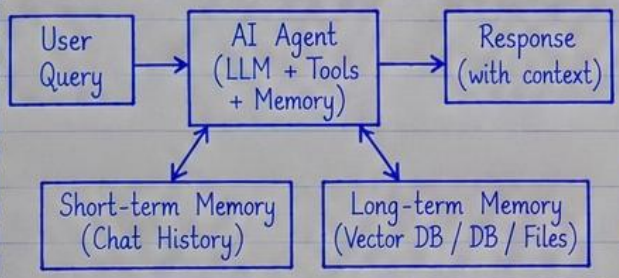
6.2 Types of Memory

① **Short-term Memory**
- Stores recent conversation context.
- Used during current session.
- Example: Chat history (last few turns).

② **Long-term Memory**
- Stores important information permanently,
- Used across sessions.
- Example: User preferences, learned facts.

③ **Vector Memory (Semantic Memory)**
- Stores information as embeddings.
- Helps in semantic search and retrieval.
- Example: Using vector database (e.g. Chroma).

6.3 Memory Architecture



6.4 Using Conversation Memory (LangChain)

```
from langchain.llms import OpenAI
from langchain.memory import ConversationBufferMemory
from langchain.agents import initialize_agent, Tool, AgentType

llm = OpenAI(model="gpt-3.5-turbo", temperature=0)
memory = ConversationBufferMemory(
        memory_key="chat_history",
        return_messages=True
)

# Initialize agent with memory
agent = initialize_agent(
        tools=[],
        llm=llm,
        agent=AgentType.CONVERSATIONAL_REACT_DESCRIPTION,
        memory=memory,
)       verbose=True
```

6.5 Using Vector Memory (Chroma)

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.memory import VectorStoreRetrieverMemory

# Create embeddings and vector store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma(
        collection_name="agent_memory",
        embedding_function=embeddings
)
memory = VectorStoreRetrieverMemory(
        retriever=vectorstore.as_retriever(),
        memory_key="long_term_memory"
)
```

6.6 Example: Personalized Travel Agent

```
from langchain.tools import Tool

# Agent with both short-term and long-term memory
agent = initialize_agent(
        tools=[search_tool],
        llm=llm,
        agent=AgentType.CONVERSATIONAL_REACT_DESCRIPTION,
        memory=memory,  # short-term
        verbose=True
)
# User can ask follow-up questions and agent
# remembers previous preferences.
```

6.7 Sample Conversation (Output)

User: I want to visit Goa next month.
Agent: Sure! I can help you plan a trip to Goa. ...

User: What about flight options?
Agent: Here are some flight options ...

User: Also remember that I prefer beach resorts.
Agent: Got it! I will remember your preference for beach resorts and suggest similar options in future.

6.8 Key Points

- → Memory improves conversation quality.
- → Use short-term memory for current session.
- → Use long-term memory for persistent information.
- → Vector memory helps in semantic search.
- → Combine memory with tools for better results.
- → Useful for personalized and multi-turn tasks.

Rahul Yadav

7. Tools and Function Calling in AI Agents

7.1 Why Tools are Important ?

- → LLM alone can only generate text.
- → Tools allow agents to take real actions.
- → Can access real-time data, APIs, databases, files, calculators etc.
- → Makes agents more useful, reliable and up-to-date.
- → Required for real-world applications.

7.2 Types of Tools

1. API Tools → Call external APIs (Weather, Flight, Maps etc.)
2. Database Tools → Query databases (SQL, MongoDB etc.)
3. File Tools → Read/Write files, documents.
4. Web Search Tools → Search latest information from internet.
5. Custom Tools → Any custom Python function.

7.3 How Function Calling Works ?



1. User gives a query.
2. LLM understands the query and decides which tool to use.
3. LLM generates a function call with required parameters.
4. The tool is executed and result is returned to LLM.
5. LLM uses the result to generate final response.

7.4 Example Tools in Python (LangChain)

1. DuckDuckGo Search Tool

```
from langchain.tools import DuckDuckGoSearchRun
search_tool = DuckDuckGoSearchRun()

# Run search
result = search_tool.run(
    "Latest news about AI Agents"
)
print(result)
```

2. Calculator Tool

```
from langchain.tools import Tool
def calculate(expression: str) → str:
    try:
        return str(eval(expression, {"__builtins__": {}}))
    except Exception as e:
        return f"Error: {str(e)}"

calculator_tool = Tool(
    name = "Calculator",
    func = calculate,
    description = "Useful for mathematical calculations"
)
```

3. Custom API Tool (Weather)

```
import requests
def get_weather(city: str) → str:
    url = f"https://api.openweathermarp.org/data/2.5/
    weather?q={city}&appid=YOUR_API_KEY&units=metric"
    response = requests.get(url)
    data = response.json()
    return f"Weather in {city}: {data['main']['temp']}°C,
                {data['weather'][0]['description']}"
weather_tool = Tool(
    name = "Weather",
    func = get_weather,
    description = "Get current weather information
                for a city"
)
```

7.5 Using Tools with an Agent

```
from langchain.agents import initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI

tools = [search_tool, calculator_tool, weather_tool]
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
# Run the agent
response = agent.run("What is the weather in Delhi and
          also calculate 25 * 34 ?")
print(response)
```

7.6 Sample Output (Simplified)

```
User: What is the weather in Delhi and also
      calculate 25 * 34 ?
Agent: I will use the weather tool to get the
       current weather in Delhi.
Agent: Weather in Delhi: 32°C, clear sky.
Agent: Now I will use the calculator tool.
Agent: 25 * 34 = 850.
Final Answer: Delhi weather is 32°C (clear sky)
              and 25 * 34 = 850.
```

7.7 Key Points

- → Tools extend the capabilities of AI agents.
- → LLM decides which tool to use based on the query.
- → Function calling allows structured tool execution.
- → We can create custom tools for any use case.
- → Useful for real-time, accurate and actionable responses.

Rahul Yadav

8. Planning and Decision Making in AI Agents

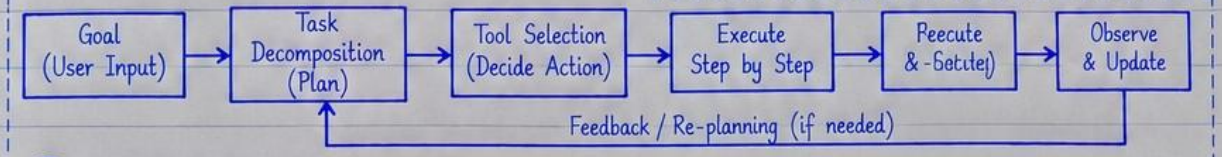
8.1 What is Planning in AI Agents?

- → Planning helps the agent break a complex goal into smaller steps to achieve it.
- → LLM alone may not always give the correct sequence of actions.
- → Planning enables better decision making, tool selection and efficient execution.
- → Useful for multi-step tasks and real-world applications (e.g. travel planning, research, data analysis etc.).

8.2 Types of Planning

1. **Zero-shot Planning**
   - LLM directly decides the next step.
   - Good for simple tasks.
2. **Few-shot Planning**
   - Provide examples to improve planning.
   - Works better for specific use cases.
3. **ReAct (Reasoning + Acting)**
   - LLM reasons about the problem, decides an action, observes result and repeats.
   - Widely used for tool-based agents.
4. **Plan-and-Execute**
   - First create a full plan and then execute step by step.
   - Better for complex tasks.

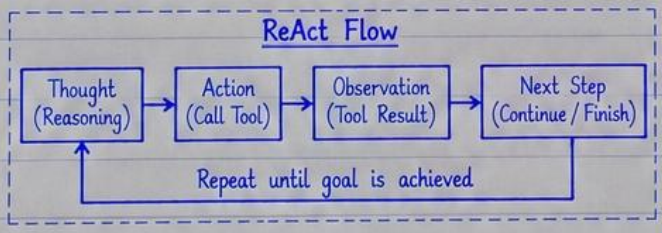
8.3 Planning Workflow



1. Understand the goal and context.
2. Break the goal into smaller steps (task decomposition).
3. Select appropriate tools for each step.
4. Execute each step and observe results.
5. Re-plan if needed and generate final response.

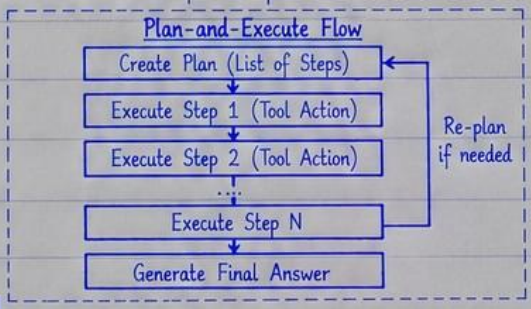
8.4 ReAct Pattern (Reasoning + Acting)

- → LLM thinks (reasoning), takes an action (tool use), observes the result and continues the loop.
- → Helps agent make better decisions.
- → Commonly used in LangChain agents.



8.5 Plan-and-Execute Pattern

- → LLM first creates a complete plan.
- → Then each step is executed using tools.
- → More stable for complex tasks.



8.6 Example: Travel Planning Agent (LangChain)

```
from langchain.agents import initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI
from langchain.tools import Tool
# Sample tools (dummy functions)
def search_flights(destination: str) → str:
    return f"Found flight options to {destination}"

def search_hotels(destination: str) → str:
    return f"Found hotel options in {destination}"

tools = [
    Tool(name="FlightSearch", func=search_flights,
         description="Search flights for a destination"),
    Tool(name="HotelSearch", func=search_hotels,
         description="Search hotels for a destination")
]
```

```
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
agent = initialize_agent(
        tools=tools,
        llm=llm,
        agent=AgentType.REACT_DESCRIPTION,
        verbose=True
)
query = "Plan a 3 day trip to Goa with flight and hotel details."
response = agent.run(query)
print(response)
```

8.7 Key Points

- → Planning helps in complex and multi-step tasks.
- → ReAct and Plan-and-Execute are popular approaches.
- → Agents can re-plan based on observation results.
- → Tool selection and step-wise execution improves accuracy.
- → Useful for real-world applications like travel planning, research, data analysis, automation etc.

Rahul Yadav

9. Multi-Agent Systems

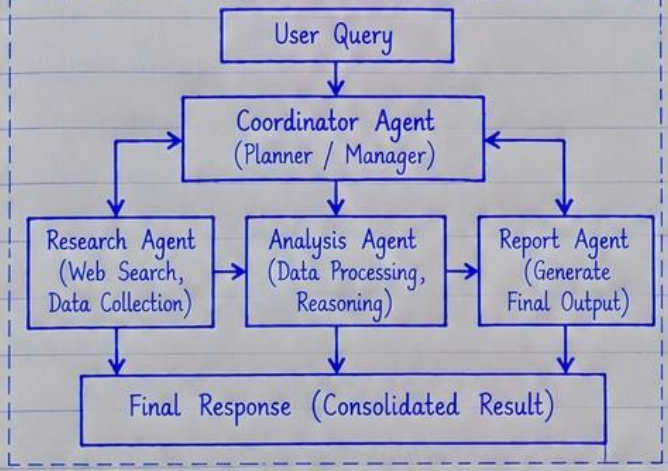
9.1 What are Multi-Agent Systems ?

- → Multiple AI agents work together to solve a complex task.
- → Each agent has a specific role and expertise.
- → Agents can communicate, collaborate and share information.
- → Useful for complex real-world problems like research, data analysis, content creation, travel planning etc.

9.2 Why Use Multi-Agent Systems ?

- → Divide complex problems into smaller tasks.
- → Each agent can specialize in a specific area.
- → Better results through collaboration.
- → Can handle multi-step workflows efficiently.
- → More scalable and flexible.
- → Useful for real-world enterprise applications.

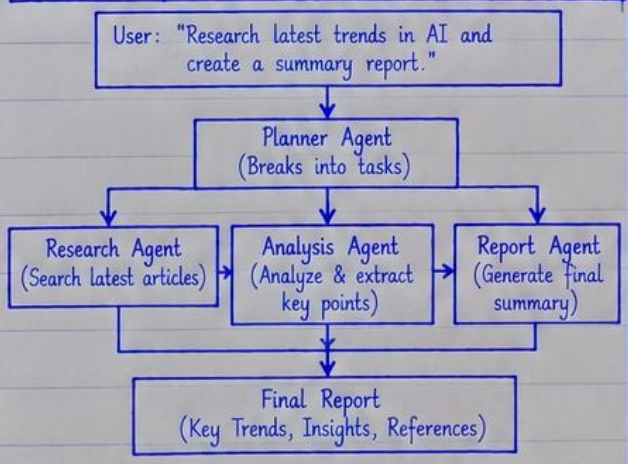
9.3 Architecture of Multi-Agent System



9.4 Common Agent Roles

1. **Planner Agent**
   - Understands goal, breaks into tasks
   - Assigns tasks to other agents
2. **Research Agent**
   - Searches web, databases, documents
   - Collects relevant information
3. **Analysis Agent**
   - Processes and analyzes the information
   - Extracts key insights
4. **Execution Agent**
   - Performs specific actions (API calls, code etc.)
   - Executes assigned tasks
5. **Critic Agent**
   - Validates the results
   - Provides feedback and improvements
6. **Reporting Agent**
   - Combines all results
   - Generates final response in a structured format

9.5 Example: Research Assistant Multi-Agent



9.6 Implementation with LangChain

```
from langchain.agents import initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI
from langchain.tools import Tool
from langchain.agents import AgentExecutor

# Create tools for different agents
def search_tool(query: str) → str:
    return "Search results for: " + query

def analysis_tool(data: str) → str:
    return "Analyzed insights from data: " + data

def report_tool(insights: str) → str:
    return "Final Report: " + insights

tools = [
    Tool(name="SearchTool", func=search_tool, description="Search information"),
    Tool(name="AnalysisTool", func=analysis_tool, description="Analyze data"),
    Tool(name="ReportTool", func=report_tool, description="Generate final report")
]
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
agent = initialize_agent(
        tools=tools,
        llm=llm,
        agent=AgentType.OPENAI_FUNCTIONS,
)       verbose=True
response = agent.run("Research latest trends in AI and create a summary report.")
print(response)
```

9.7 Sample Output (Simplified)

```
User: Research latest trends in AI and create a summary report.
Planner Agent: Breaking the task into steps...
Research Agent: Searching latest articles...
Analysis Agent: Extracting key insights...
Report Agent: Generating final report...

Final Output:
  1. Trend 1: Growth of Generative AI
  2. Trend 2: AI Agents in Enterprise
  3. Trend 3: Multimodal Models
  4. Key Takeaways and Future Outlook ...
```

9.8 Key Points

- → Multi-agent systems can solve complex problems.
- → Each agent has a specific role and toolset.
- → Agents collaborate to produce better results.
- → LangChain makes it easy to build multi-agent systems.
- → Useful for research, automation, enterprise and real-world applications.
  applications.

Rahul Yadav

10. RAG (Retrieval Augmented Generation)

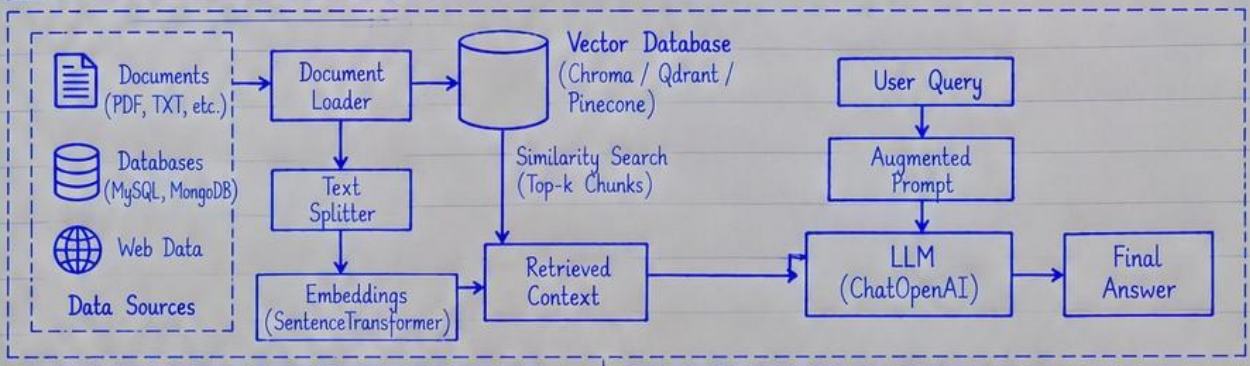
10.1 What is RAG ?

- → RAG (Retrieval Augmented Generation) combines the power of retrieval (search) with LLM generation.
- → It fetches relevant information from your private data (documents, databases, etc.) and provides accurate, up-to-date answers.
- → Helps reduce hallucinations in LLMs.
- → Useful for enterprise applications, research, chatbots, knowledge bases etc.

10.2 How RAG Works ?

1. Ingest Data → Load documents and split into smaller chunks.
2. Create Embeddings → Convert each chunk into vector embeddings.
3. Store in Vector DB → Save embeddings in vector database (e.g. Chroma, Qdrant, Pinecone).
4. Retrieve Relevant Chunks → Find top-k relevant chunks for user query.
5. Augment Prompt → Add retrieved context to the LLM prompt.
6. Generate Answer → LLM generates a final answer using retrieved context.

10.3 RAG Architecture



10.4 Data Ingestion with LangChain

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

# Load PDF document
loader = PyPDFLoader("data/ai_notes.pdf")
documents = loader.load()

# Split documents into chunks
splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200
)
docs = splitter.split_documents(documents)

# Create embeddings and store in Chroma
embeddings = OpenAIEmbeddings()
vectordb = Chroma.from_documents(
        documents=docs,
        embedding=embeddings,
        persist_directory="./chroma_db"
)
```

10.5 Retrieval and QA Chain

```
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
# Load existing vector database
vectordb = Chroma(
        persist_directory="./chroma_db",
        embedding_function=OpenAIEmbeddings()
)
# Create retriever
retriever = vectordb.as_retriever(search_type="similarity",
                                  search_kwargs={"k": 3})
# Create RAG chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever,
        return_source_documents=True
)
```

10.6 Sample Query and Output

```
query = "What are the key concepts of AI Agents?"
response = qa_chain.run(query)
print(response)
```

Output:
AI Agents are systems that can perceive, reason, and take actions using tools and memory. Key concepts include planning, memory, tool usage, and goal-oriented behavior...
(with sources from your documents)

10.7 Popular Tools for RAG

1. LangChain → Easy RAG pipeline and integrations.
2. LlamaIndex → Simple data ingestion and querying.
3. Chroma → Lightweight vector database (local).
4. Qdrant → High performance vector database.
5. Pinecone → Managed vector database (cloud).
6. OpenAI / Ollama → LLM for generation.

10.8 Key Points

- → RAG combines retrieval with generation.
- → Use good chunking strategy for better results.
- → Vector database helps in fast similarity search.
- → RAG reduces hallucinations and provides accurate answers.
- → Useful for chatbots, document Q&A, research and enterprise apps.

Rahul Yadav

11. Agent Frameworks and Ecosystem

11.1 What are Agent Frameworks?

- → Agent frameworks provide ready-to-use building blocks to create AI agents.
- → They handle common tasks like memory, tool calling, planning, multi-agent collaboration etc.
- → Save development time and provide standardized interfaces.
- → Allow developers to focus on business logic instead of low-level implementation.
- → Wide ecosystem with open-source and commercial options.

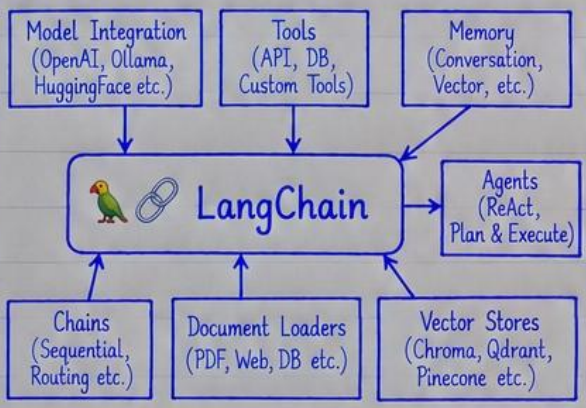
11.2 Popular Agent Frameworks

1. LangChain → Most popular, flexible and large ecosystem.
2. LlamaIndex → Best for data ingestion and RAG.
3. Microsoft AutoGen → Multi-agent conversation framework.
4. CrewAI → Simple and powerful multi-agent framework.
5. Semantic Kernel → Developed by Microsoft, good for enterprise.
6. LangGraph → Build stateful and complex agent graphs.
7. Haystack → Great for search and RAG pipelines.
8. OpenAI Assistants → Easy to use, managed service.
9. Custom Frameworks → Build your own for specific needs.

11.3 Framework Comparison

Framework	Best For	Key Features	Language Support	Use Case
LangChain	General purpose	Tools, memory, chains, agents	Python, JS	Chatbots, RAG, agents
LlamaIndex	Data + RAG	Data ingestion, indexing	Python	Knowledge base, RAG
AutoGen	Multi-agent	Conversation, agent collaboration	Python	Multi-agent systems
CrewAI	Multi-agent	Simple API, role-based agents	Python	Task automation
Semantic Kernel	Enterprise	Tool calling, planning, memory	Python, C#, JS	Enterprise AI agents
LangGraph	Stateful agents	Graph-based workflows	Python	Complex agent workflows
Haystack	Search + RAG	Document search, pipelines	Python	QA systems, RAG
OpenAI Assistants	Managed service	Built-in tools, vector store	Python	Quick agent development

11.4 LangChain Ecosystem



11.5 Example: Using CrewAI (Multi-Agent)

```
from crewai import Agent, Task, Crew, Process
from langchain.chat_models import ChatOpenAI

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# Create agents
researcher = Agent(
    role="Researcher",
    goal="Find latest AI news",
    backstory="You are an expert researcher.",
    llm=llm
)
writer = Agent(
    role="Writer",
    goal="Write a summary",
    backstory="You are a technical content writer.",
)    llm=llm

# Define tasks
task1 = Task(description="Search latest AI news", agent=researcher)
task2 = Task(description="Write a summary of the findings", agent=writer)

# Create crew
crew = Crew(agents=[researcher, writer], tasks=[task1, task2],
            process=Process.sequential)
# Run
result = crew.kickoff()
print(result)
```

11.6 Other Ecosystem Tools

- OpenAI → LLM models, Assistants API and tool support
- Hugging Face → Open source models and datasets
- Pinecone → Managed vector database
- Qdrant → High performance vector database
- Chroma → Lightweight local vector database
- Ollama → Run open source models locally

11.7 Key Points

- → Agent frameworks simplify agent development.
- → Choose a framework based on your use case (RAG, multi-agent, enterprise etc.).
- → LangChain and LlamaIndex are great for RAG applications.
- → CrewAI, AutoGen and LangGraph are powerful for multi-agent systems.
- → OpenAI Assistants provide a managed and easy way to build agents.
- → Ecosystem tools like vector databases, model providers and APIs enhance agent capabilities.

Rahul Yadav

12. Building a Complete RAG Application

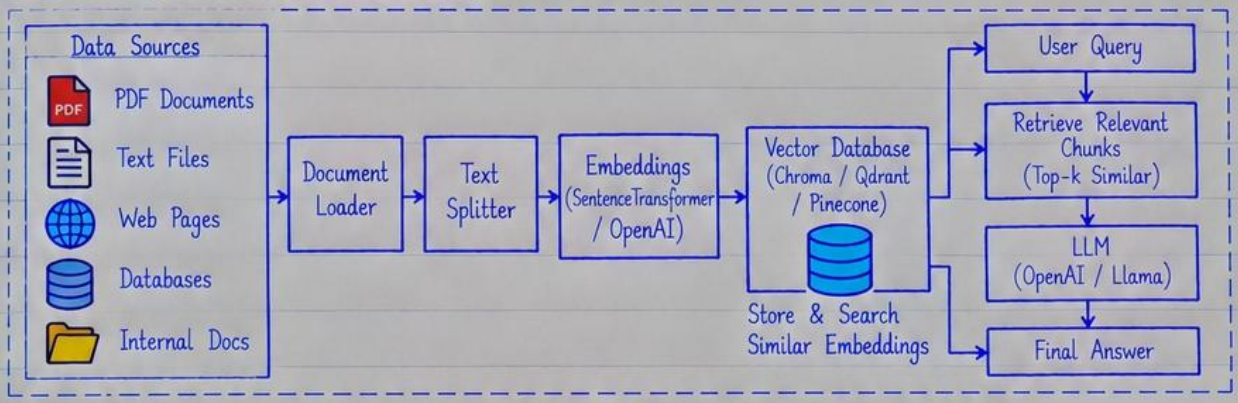
12.1 What is RAG Application ?

- → A RAG application retrieves relevant information from your private data and uses an LLM to generate accurate, context-aware answers.
- → Combines retrieval (search) + generation (LLM).
- → Useful for documents, FAQs, knowledge bases, internal data etc.
- → Reduces hallucinations and provides reliable, up-to-date answers.
- → Widely used in chatbots, enterprise search, research, support systems etc.

12.2 Complete RAG Workflow

1. Ingest Documents → Load PDFs, text files, web pages etc.
3. Split into Chunks → Break documents into smaller chunks.
3. Create Embeddings → Convert chunks into vector embeddings.
4. Store in Vector DB → Save embeddings in vector database (Chroma, Qdrant etc).
5. Retrieve Relevant Chunks → Search similar chunks for user query.
6. Generate Answer → LLM generates a final answer using retrieved context.

12.3 High Level Architecture



12.4 Step-by-Step Implementation (Python)

```
# 1. Install required libraries
pip install langchain langchain-community \
    chromadb sentence-transformers openai pypdf

# 2. Load and split document
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("data/ai_notes.pdf")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)

# 3. Create embeddings and store in Chroma
from langchain_community.embeddings import
    SentenceTransformerEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = SentenceTransformerEmbeddings(
    model_name="all-MiniLM-L6-v2"
)
vectorstore = Chroma.from_documents(
                documents=chunks,
                embedding=embeddings,
                persist_directory="./chroma_db"
)
```

12.5 Create RAG Chain

```
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
# Load vector store
vectorstore = Chroma(
      persist_directory="./chroma_db",
      embedding_function=embeddings
)
# Create retriever
retriever = vectorstore.as_retriever(search_type="similarity",
                                     search_kwargs={"k": 3})

# Create RAG chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
rag_chain = RetrievalQA.from_chain_type(
      llm=llm,
      retriever=retriever,
)     return_source_documents=True
```

12.6 Query and Response

```
query = "What are the key concepts of AI Agents?"
result = rag_chain.run(query)
print(result)

# You can also get source documents
response = rag_chain({"query": query})
print(response["source_documents"])
```

12.7 Key Points

- → Ingest, split, embed, store and retrieve relevant chunks.
- → Use vector databases like Chroma, Qdrant or Pinecone.
- → Choose the right chunk size and embedding model.
- → RAG provides accurate, up-to-date and context-aware answers.
- → Useful for chatbots, enterprise knowledge bases and research.

Rahul Yadav

13. Vector Databases for RAG

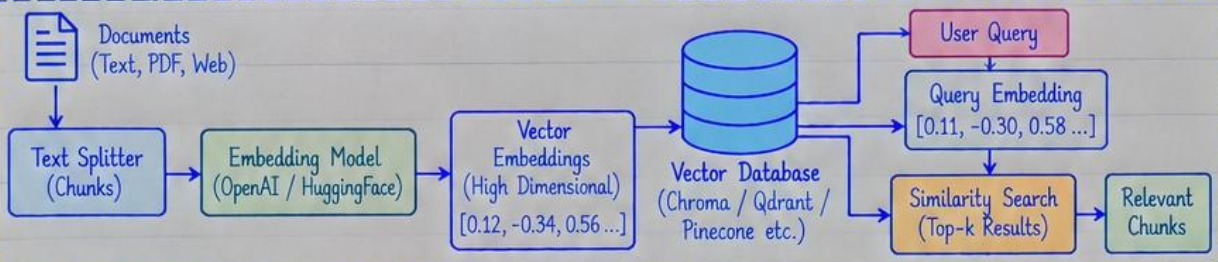
13.1 What is a Vector Database ?

- → A vector database stores and searches high-dimensional vector embeddings efficiently.
- → It helps in finding similar data using vector similarity search (semantic search).
- → Essential for RAG applications to retrieve relevant context from large documents.
- → Examples: Chroma, Pinecone, Qdrant, Weaviate, Milvus, FAISS.

13.2 Why Use a Vector Database ?

- → Fast similarity search on embeddings.
- → Handles large unstructured data (text, images, etc.).
- → Supports metadata filtering.
- → Scalable and optimized for AI applications.
- → Much better than traditional databases for semantic search use cases.

13.3 How Vector Search Works ?



13.4 Popular Vector Databases

Database	Type	Key Features	Best For
Chroma	Open Source	Simple, easy to use, local setup	Small to medium projects
Pinecone	Managed (Cloud)	High performance, scalable	Production applications
Qdrant	Open Source / Managed	Filtering, payload, high performance	Enterprise applications
Weaviate	Open Source	GraphQL, modular	Advanced use cases
Milvus	Open Source	High performance, large scale	Large datasets
FAISS	Open Source (Meta)	Fast similarity search	Research / Offline use

13.5 Similarity Search

- → Vectors are compared using distance metrics like:
  - Cosine Similarity (most common)
  - Euclidean Distance
  - Dot Product
- → Returns top-k most similar vectors (chunks).
- → Helps LLM get relevant context for generation.

13.6 Distance Metrics

Metric	Description	Use Case
Cosine Similarity	Measures angle between vectors	Text embeddings (most common)
Euclidean Distance	Straight line distance	General purpose
Dot Product	Measures vector similarity	Neural models

13.7 Example: Using Chroma (Python)

```
# Install library
pip install chromadb sentence-transformers

# Load documents
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("data/ai_notes.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)

# Create embeddings
from langchain_community.embeddings import SentenceTransformerEmbeddings
embeddings = SentenceTransformerEmbeddings(
        model_name="all-MiniLM-L6-v2"
)

# Store in Chroma
from langchain_community.vectorstores import Chroma
vectordb = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory="./chroma_db"
)
```

13.8 Query from Chroma

```
query = "What are the key concepts of AI Agents?"
docs = vectordb.similarity_search(query, k=3)

for i, doc in enumerate(docs):
    print(f"Result {i+1}:")
    print(doc.page_content)
    print("Source:", doc.metadata.get("source"))
```

13.9 Key Points

- → Vector databases store embeddings for semantic search.
- → Use embedding models like OpenAI, HuggingFace etc.
- → Supports similarity search with efficient indexing.
- → Choose the right vector DB based on project size and requirements.
- → Essential component in RAG applications.
- → Helps LLM get relevant, accurate and up-to-date information from private data.

Rahul Yadav

14. Advanced RAG Techniques

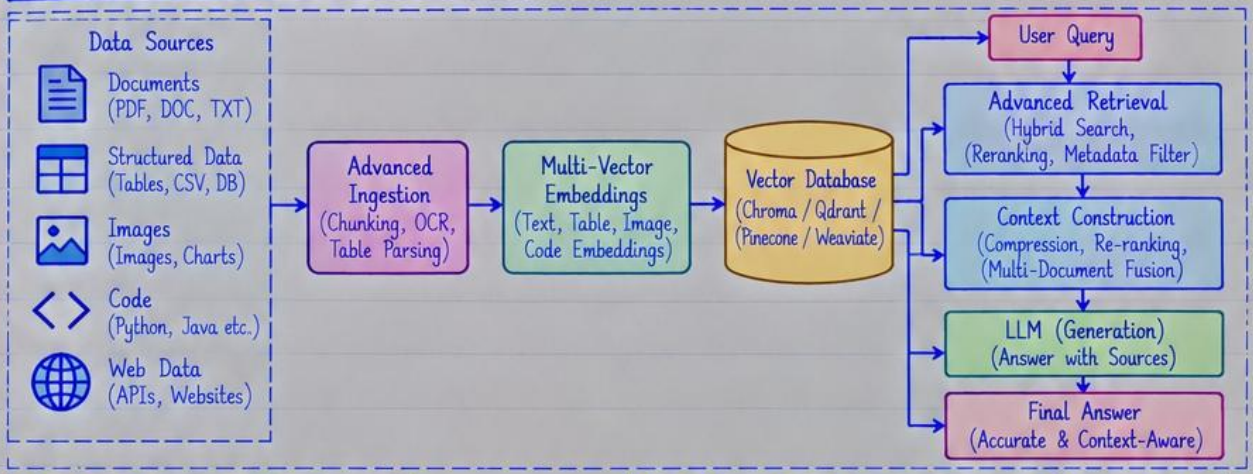
14.1 Why Advanced RAG ?

- → Basic RAG works well for simple use cases but has limitations.
- → Advanced techniques improve accuracy, relevance and robustness.
- → Helps handle real-world data (large docs, tables, images, multi-modal).
- → Reduces hallucinations and provides better context understanding.
- → Useful for enterprise applications and production systems.

14.2 Common Challenges in Basic RAG

1. Retrieval Quality → Sometimes retrieves irrelevant chunks.
2. Context Length → Important information may be missed.
3. Query Understanding → Complex queries are not handled well.
4. Data Diversity → Tables, images, code, structured data.
5. Hallucinations → LLM may still generate incorrect answers.
6. Scalability → Handling large datasets efficiently.
7. Evaluation → Hard to measure answer quality.

14.3 Advanced RAG Architecture



14.4 Advanced Techniques

1. **Hybrid Search** — Combine semantic search (vector) + keyword search (BM25).
2. **Reranking** — Use cross-encoder or LLM rerankers to improve relevance.
3. **Context Compression** — Reduce token size by summarizing retrieved chunks.
4. **Multi-Vector Retrieval** — Use different embeddings for text, tables, images, code.
5. **Metadata Filtering** — Filter results based on metadata (date, source, category).
6. **Query Expansion** — Rewrite or expand user queries for better retrieval.
7. **Multi-Modal RAG** — Handle text, images, tables and code together.
8. **Self-Querying** — LLM decides which data source and filters to use.
9. **Evaluation & Feedback** — Use metrics and user feedback to improve retrieval quality.

14.5 Hybrid Search Example (LangChain)

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.retrievers import BM25Retriever, EnsembleRetriever
# Vector retriever
vector_store = Chroma(persict_directory="./chroma_db",
      embedding_function=OpenAIEmbeddings())
vector_retriever = vector_store.as_retriever(search_kwargs={"k": 3})
# Keyword retriever
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 3
# Hybrid search (combine both)
retriever = EnsembleRetriever(
    retrievers=[vector_retriever, bm25_retriever],
    weights=[0.7, 0.3]
)
results = retriever.get_relevant_documents("What is RAG?")
for doc in results:
    print(doc.page_content)
    print("Source:", doc.metadata.get("source"))
    print("---")
```

14.6 Context Compression Example

```
from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain.chat_models import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
compressor = LLMChainExtractor.from_llm(llm)
compressed_retriever = ContextualCompressionRetriever(
        base_retriever=vector_retriever,
        base_compressor=compressor
)
docs = compressed_retriever.get_relevant_documents("Explain AI Agents")
for doc in docs:
    print(doc.page_content)
```

14.7 Key Points

- → Use hybrid search for better retrieval results.
- → Apply reranking to improve relevance.
- → Compress context to fit within token limits.
- → Use multiple embeddings for text, tables, images, code.
- → Filter results using metadata.
- → Query expansion helps with complex questions.
- → Multi-modal RAG can handle different data types.
- → Always evaluate and improve retrieval quality.
- → Advanced RAG is useful for production and enterprise use cases.

Rahul Yadav

15. Multi-Modal RAG (Text + Image + Table)

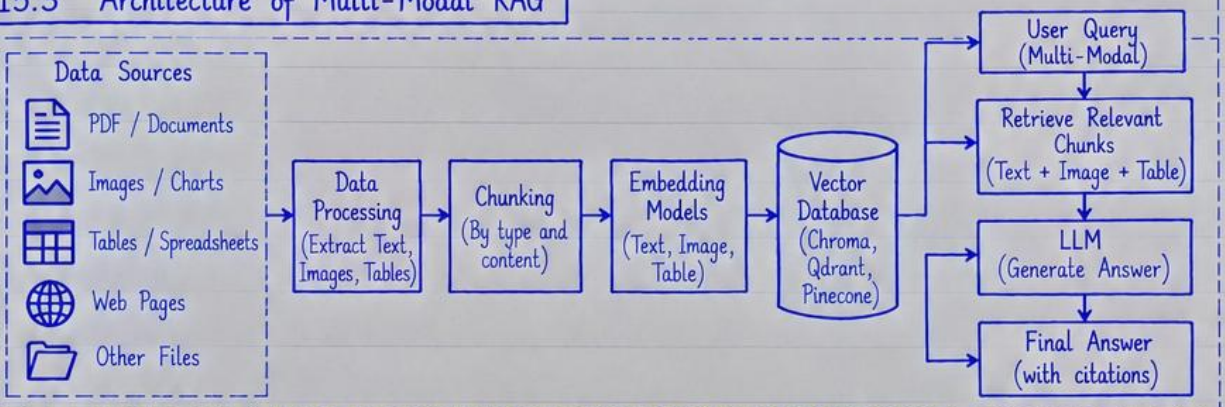
15.1 What is Multi-Modal RAG ?

- → Multi-Modal RAG can understand and retrieve information from multiple data types like text, images, tables and charts.
- → It extracts content from different formats and creates unified embeddings for search.
- → Useful for real world applications where information is available in PDFs, images, scanned documents, tables, etc.
- → Provides more accurate and context-aware answers compared to text-only RAG.

15.2 Use Cases

1. Analyze PDF reports with text, images and tables.
2. Search information from scanned documents.
3. Extract data from charts and graphs.
4. Enterprise knowledge base (manuals, documents, images).
5. Research and academic documents.
6. Medical reports (text + images).
7. Financial reports (text + tables + charts).
8. Product documentation with diagrams.

15.3 Architecture of Multi-Modal RAG



15.4 Data Processing Steps

1. Load documents (PDF, images, tables etc).
2. Extract text using PDF loaders / OCR.
3. Extract images and convert to embeddings using vision models (CLIP, OpenAI).
4. Extract tables and convert to text or structured format (CSV/Markdown).
5. Create chunks for each type (text, image, table).
6. Generate embeddings and store in vector database.

15.5 Example: Loading PDF with Images and Tables

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders import UnstructuredPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load PDF with images and tables
loader = UnstructuredPDFLoader("data/report.pdf", mode="elements")
docs = loader.load()

# Split documents
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)

print(f"Total chunks: {len(chunks)}")
```

15.6 Image Embeddings (CLIP)

```
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma

# Use CLIP model for image embeddings
from langchain_community.embeddings import HuggingFaceEmbeddings
image_embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/clip-ViT-B-32"
)
# Create vector store
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=image_embeddings
)

print("Image embedding model loaded successfully!")
```

15.7 Query with Multi-Modal Input

```
from langchain.retrievers import MultiVectorRetriever
from langchain.chat_models import ChatOpenAI
retriever = MultiVectorRetriever(vectorstore=vectorstore)
llm = ChatOpenAI(model="gpt-4o", temperature=0)
query = "What information is shown in this chart?"
results = retriever.get_relevant_documents(query)
response = llm.invoke(
    f"Answer the question based on the retrieved content:\n{query}\n"
)
print(response.content)
```

15.8 Key Points

- → Multi-Modal RAG supports text, images, tables and charts.
- → Use appropriate models for each data type (text, vision, table).
- → Extract and chunk content based on data type.
- → Store embeddings in a vector database like Chroma, Qdrant or Pinecone.
- → Provides better and more accurate answers for complex documents.
- → Widely used in enterprise, research, medical, financial and education domains.

Rahul Yadav

16. RAG Evaluation and Optimization

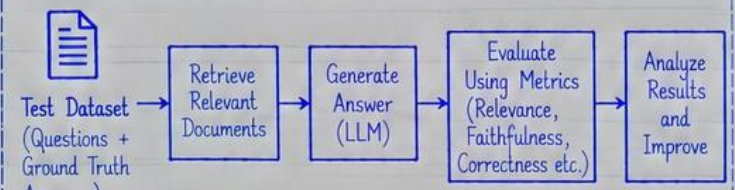
16.1 Why Evaluate RAG ?

- → To ensure the generated answers are accurate, relevant and grounded in the provided documents.
- → Helps identify issues like hallucinations, irrelevant results and poor retrieval.
- → Important for production use cases where reliability and trust are critical.
- → Evaluation helps to compare different models, embeddings, chunking strategies and retrieval techniques.
- → Continuous evaluation ensures improvement over time as new data is added.

16.2 Key Evaluation Metrics

1. Context Relevance → How relevant are retrieved chunks to the query.
2. Answer Relevance → How relevant is the final answer to the query.
3. Faithfulness → Whether the answer is grounded in the retrieved context.
4. Hallucination Rate → Measures how often the model generates incorrect information.
5. Retrieval Recall → How many relevant documents are retrieved.
6. Retrieval Precision → How many retrieved documents are actually relevant.
7. Answer Correctness → How factually correct is the answer.
8. Latency → Time taken for end-to-end response.

16.3 RAG Evaluation Process



16.4 Evaluation Datasets

- → Create a set of test questions and ground truth answers.
- → Include different types of questions:
  - Factual questions
  - Analytical questions
  - Multi-hop questions
  - Domain specific questions
  - Negative / out-of-scope questions
- → Use real user queries for better evaluation.

16.5 Example: Evaluate RAG with LangChain (Ragas)

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA
from ragas import evaluate
from ragas.metrics import (
    context_relevance,
    answer_relevance,
    faithfulness,
    answer_correctness
)
# Prepare test data
test_data = [
  { "question": "What is RAG?",
    "ground_truth": "RAG retrieves relevant documents and uses
     them to generate answers using an LLM."},
 ]
```

```
# Run RAG pipeline
loader = PyPDFLoader("data/ai_notes.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)
retriever = vectorstore.as_retriever(search_kwargs={"k":3})
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-3.5-turbo", temperature=0),
    retriever=retriever
)
# Evaluate
results = evaluate(
  qa_chain,
  test_data,
  metrics=[context_relevance, answer_relevance, faithfulness, answer_correctness]
)
print(results)
```

16.6 Optimization Techniques

1. Better Chunking → Use meaningful chunk size and overlap.
2. Metadata Filtering → Add metadata like source, date, category.
3. Hybrid Search → Combine vector search with keyword search (BM25).
4. Reranking → Use cross-encoder or LLM reranker.
5. Query Reformulation → Rewrite complex queries for better retrieval.
6. Context Compression → Remove irrelevant information from chunks.
7. Multi-Query Retrieval → Generate multiple query variations.
8. Fine-tuning / Prompting → Use better prompts or fine-tuned models.

16.7 Common Issues and Solutions

Issue	Solution
Irrelevant results	Improve chunking, use metadata filtering, reranking.
Missing context	Increase top-k, use multi-query retrieval.
Hallucinations	Use grounded prompting, verify with citations.
Slow response	Use smaller models, optimize vector DB, cache results.
Low accuracy	Use better embeddings, reranking, hybrid search.
Outdated information	Regularly update your data and re-index.

16.8 Key Points

- → Evaluation is essential for building reliable RAG applications.
- → Use metrics like relevance, faithfulness and correctness.
- → Continuously optimize with better chunking, search and reranking.
- → Test with real-world queries and edge cases.
- → Monitor performance in production and keep improving.

Rahul Yadav

17. RAG in Production (Best Practices)

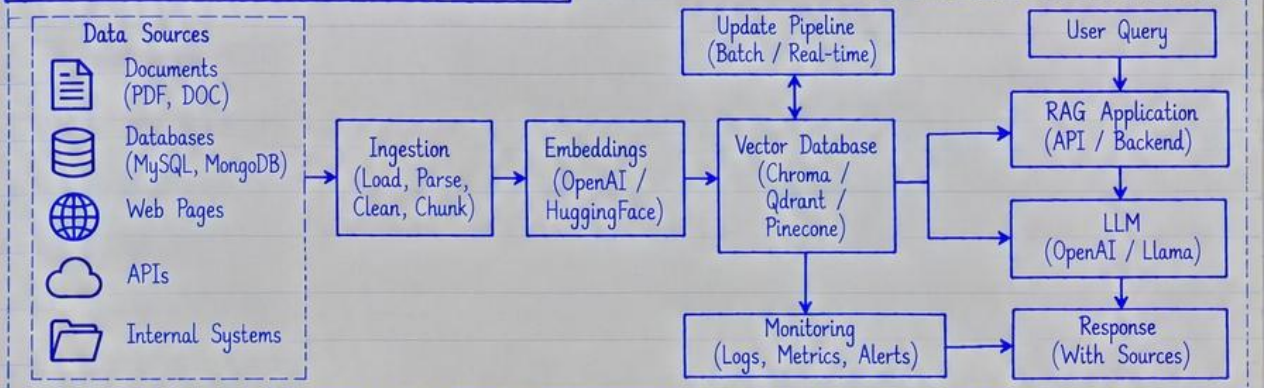
17.1 Why Productionization is Important ?

- → Move from prototype to real-world usage.
- → Handle large scale data and users.
- → Ensure accuracy, performance, security and reliability.
- → Monitor and improve continuously.
- → Required for enterprise and customer-facing applications.

17.2 Key Considerations

1. Data Ingestion and Updates (batch / real-time)
2. Scalable and Reliable Vector Database
3. Efficient Chunking and Embeddings
4. Prompt Engineering and Query Handling
5. Caching and Latency Optimization
6. Monitoring and Observability
7. Security and Access Control
8. Cost Optimization

17.3 Production RAG Architecture



17.4 Data Ingestion Best Practices

1. Support multiple formats (PDF, DOC, TXT, CSV, Images).
2. Use robust document loaders.
3. Clean and preprocess data (remove noise, headers, footers).
4. Split into meaningful chunks (semantic chunking).
5. Maintain metadata (source, date, category, access level).
6. Automate updates using scheduled jobs or webhooks.
7. Handle incremental updates (only process new/changed data).

17.5 Chunking Strategies

1. Fixed Size Chunking
   - → Simple and effective for most use cases.
2. Recursive Chunking
   - → Split based on headings, sections, structure.
3. Semantic Chunking
   - → Use embeddings to create semantically meaningful chunks.
4. Overlap
   - → Add overlap (e.g., 100–200 tokens) to maintain context.
5. Metadata
   - → Store metadata like source, page number, section etc.

17.6 Embedding and Vector DB Tips

- → Use high-quality embedding models (OpenAI, HuggingFace).
- → Normalize and clean text before embedding.
- → Choose the right vector database based on your needs:
  - Chroma → Easy to use, local setup
  - Qdrant → High performance, good filtering
  - Pinecone → Managed, scalable
  - Weaviate → GraphQL, hybrid search
- → Use metadata filtering for better search results.
- → Regularly re-index updated data.
- → Monitor vector DB size and performance.

17.7 Example: Incremental Ingestion (Python)

```python
from langchain_community.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
import os, time

# Load new or updated documents
loader = DirectoryLoader("./data", glob="*.pdf")
docs = loader.load()

# Split documents
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)

# Create embeddings and store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)
print("Ingestion completed successfully!")
```

17.8 Monitoring and Observability

- → Log user queries and retrieved documents.
- → Track metrics: response time, relevance score, error rate.
- → Use tools like Prometheus + Grafana, LangSmith, or custom logs.
- → Monitor vector DB size, latency and usage.
- → Set up alerts for failures and performance issues.

17.9 Key Points

- → Production RAG requires robust data pipeline, monitoring and security.
- → Use efficient chunking, embeddings and vector database.
- → Optimize for latency, cost and accuracy.
- → Continuously monitor and improve based on real user feedback.

Rahul Yadav

18. Future Trends and Advanced Topics in RAG

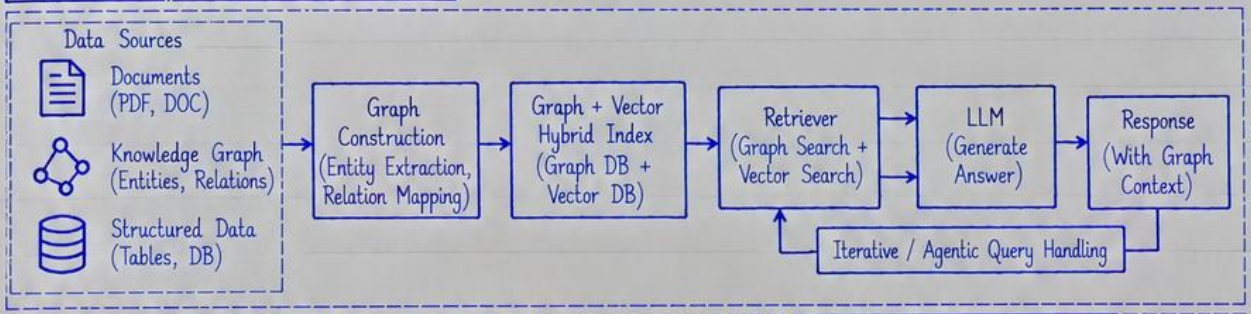
18.1 Emerging Trends in RAG

- → Multi-Modal RAG (Text + Image + Video + Audio)
- → Graph RAG (Knowledge Graphs)
- → Agentic RAG (Tool use, multi-step reasoning)
- → Personalized RAG (User-specific knowledge)
- → Real-time / Streaming RAG
- → Domain-specific RAG (Healthcare, Finance, Legal)
- → RAG for structured data (SQL + Vector)
- → Edge / On-device RAG
- → Open-source LLMs with RAG (Llama, Mistral, Phi)

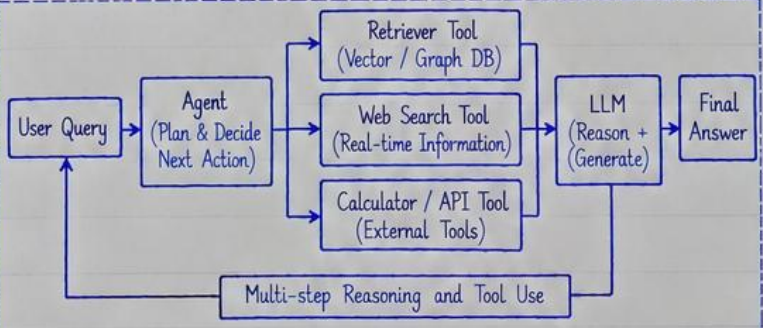
18.2 Advanced RAG Techniques

1. Graph-based Retrieval → Use knowledge graphs to capture relationships and improve context.
2. Agentic RAG → Use AI agents to decide what, when and how to retrieve.
3. Self-Query RAG → LLM generates structured queries (e.g., filters, SQL) for better retrieval.
4. Routing RAG → Route query to different data sources based on intent.
5. Re-Ranking with LLM → Use cross-encoder or LLM for better ranking.
6. Iterative Retrieval → Multi-step retrieval and refinement.
7. Knowledge Distillation → Create smaller domain-specific models for cost-efficient RAG.

18.3 Graph RAG Architecture



18.4 Agentic RAG (RAG with Tools)



18.5 Self-Query RAG

- → LLM converts natural language query into structured query.
- → Can generate filters, SQL, or metadata conditions.
- → Helps retrieve more relevant documents.
- → Useful for structured and semi-structured data (databases, tables).

Example

User: "Show me sales reports for Q1 2024 from India."

LLM generates:

- filter: {"country": "India", "quarter": "Q1-2024"}
- or SQL: SELECT * FROM sales WHERE country = 'India' AND quarter = 'Q1-2024'

18.6 RAG with Structured Data (Text + SQL)

- → Combine vector search with SQL queries.
- → Useful when data is stored in relational databases.
- → Approach:

1. Convert natural language to SQL (Text-to-SQL).
2. Execute query on database.
3. Combine results with unstructured documents.
4. Generate final answer using LLM.

18.7 Challenges and Future Solutions

Challenge	Future Solution
Large-scale data	Better indexing, distributed vector DBs
Real-time data	Streaming ingestion, incremental updates
Multi-modal data	Unified embeddings and cross-modal models
Hallucinations	Stronger grounding, verification with tools
Complex queries	Agentic RAG, self-query, graph RAG
Cost and latency	Smaller models, model distillation, caching
Domain adaptation	Domain-specific embeddings and fine-tuning

18.8 Practical Use Cases (Future)

- → Enterprise knowledge assistants (multi-modal)
- → Research assistants with web + local data
- → AI agents for business workflows
- → Healthcare diagnosis with medical knowledge graphs
- → Legal document analysis
- → Financial data analysis with real-time data
- → Personal AI companion with user data (emails, files, notes)
- → Automation and autonomous agents

18.9 Key Points

- → RAG is rapidly evolving with new techniques and tools.
- → Graph RAG and Agentic RAG can handle complex real-world queries.
- → Multi-modal and structured data RAG are important for future applications.
- → Use the right approach based on your domain and data type.
- → Focus on accuracy, reliability, security and scalablity.
- → RAG will continue to be a key technology for AI applications in coming years.

Rahul Yadav

19. Complete RAG Project Example (End-to-End)

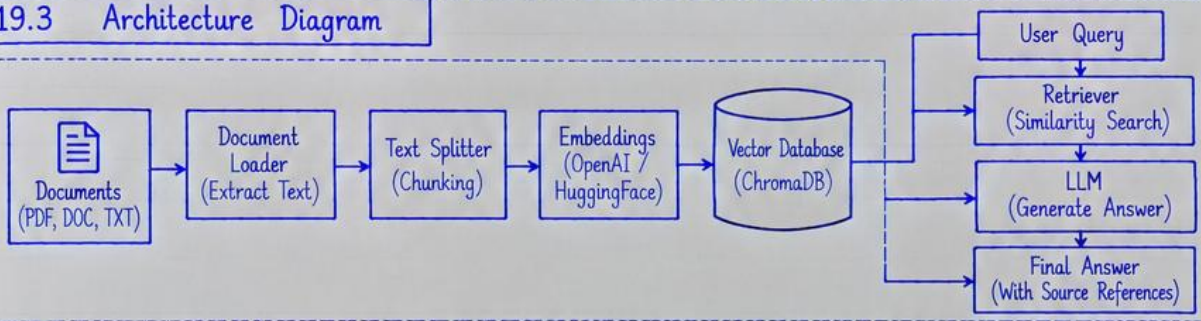
19.1 Project Overview

- → Build a complete RAG application using Python, LangChain, OpenAI (or local LLM), and ChromaDB.
- → Upload documents (PDF, DOC, TXT).
- → Extract text, split into chunks.
- → Create embeddings and store in vector DB.
- → Ask questions and get context-aware answers with source references.
- → Simple UI using Streamlit (optional).

19.2 Tech Stack

1. Python (LangChain)
2. Document Loaders (PDF, DOC, TXT)
3. Text Splitter (RecursiveCharacterTextSplitter)
4. Embeddings (OpenAIEmbeddings / HuggingFaceEmbeddings)
5. Vector Database (ChromaDB)
6. LLM (OpenAI / Local LLM)
7. Streamlit (for UI)
8. Optional: Reranking, Metadata Filtering

19.3 Architecture Diagram



19.4 Install Required Libraries

```
pip install langchain
pip install langchain-community
pip install langchain-openai
pip install langchain-chroma
pip install pypdf
pip install python-docx
pip install streamlit  # optional for UI
```

19.5 Load and Split Documents

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load PDF document
loader = PyPDFLoader("data/sample.pdf")
docs = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)
```

19.6 Create Embeddings and Store in ChromaDB

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# Create embeddings
embeddings = OpenAIEmbeddings()

# Create and store in ChromaDB
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3}
)
```

19.7 Query and Generate Answer

```
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# Create LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# Create RAG chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)
# Ask a question
query = "What is the main topic of this document?"
result = qa_chain({"query": query})
print("Answer:", result["result"])
print("Sources:", result["source_documents"])
```

19.8 Simple Streamlit UI (Optional)

```
import streamlit as st

st.title("📄 RAG Document Assistant")
query = st.text_input("Ask a question:")
if st.button("Get Answer"):
    result = qa_chain({"query": query})
    st.write("### Answer")
    st.write(result["result"])
    st.write("### Sources")
    for doc in result["source_documents"]:
        st.write(doc.metadata.get("source", "Unknown"))
```

19.9 Enhancements (Optional)

1. Support multiple file formats (PDF, DOC, TXT).
2. Add metadata filtering (date, source, category).
3. Use reranking for better results.
4. Hybrid search (vector + keyword).
5. Add authentication for private data.
6. Display source page numbers.
7. Support multi-modal (text + images).
8. Deploy as a web application.

19.10 Key Takeaways

- → RAG combines retrieval with LLM for accurate, context-aware answers.
- → Works with private and domain-specific data.
- → Can be extended with advanced techniques like reranking, graph RAG and multi-modal input.
- → Useful for real-world applications in enterprise and research.

Rahul Yadav